

Ajout de validations de contraintes à un modèle

Prolematiques

- Aller dans l'interface Room :

1- Essayer de donner un seul code pour deux chambres **Room No.**

2- Essayer de remplir le montant avec un montant négatif.

Qu'est ce que vous remarquez ??

Notre application accepte ces incohérences.

Ajout de validations de contraintes à un modèle

- Nous voulons nous assurer que nos modèles ne contiennent pas de données invalides ou incohérentes.
- Odoo propose deux types de contraintes pour cela :
 - **Contraintes au niveau de la base de données :**
 - Ce sont les contraintes prises en charge par PostgreSQL.
 - Les plus courantes sont les contraintes UNIQUE, qui empêchent les valeurs en double.
 - Nous pouvons également utiliser des contraintes **CHECK** et **EXCLUDE** pour d'autres conditions.
 - Ces contraintes sont rapides et fiables, mais elles sont limitées par les fonctionnalités de PostgreSQL.
 - **Contraintes au niveau du serveur :**
 - Ce sont des contraintes que nous écrivons en code Python.
 - Nous les utilisons lorsque les contraintes au niveau de la base de données ne suffisent pas.
 - Elles sont plus flexibles et puissantes, mais aussi plus lentes et complexes.

Objectif de ce chapitre

- 1) **Nous allons renforcer notre application avec des contraintes.**
- 2) **Nous utiliserons une contrainte UNIQUE pour garantir que les identifiants de chambre ne sont pas dupliqués.**
- 3) **Nous ajouterons également une contrainte Python pour vérifier que le montant du loyer est positif.**

1. Les contraintes SQL sont définies via l'attribut de modèle `_sql_constraints`.

- **l'attribut de modèle `_sql_constraints`.**
- **NB !: Les modifications seront effectuées dans Dans votre Modele Foyer Room**
- Cet attribut prend une liste de triplets contenant des chaînes de caractères (nom, définition_sql, message),
- **pour ajouter une contrainte d'unicité**
- où :
 - - nom est un identifiant valide pour la contrainte SQL,
 - - définition_sql est une expression de type table_constraint,
 - - message est le texte d'erreur affiché en cas de violation.
- Nous pouvons ajouter le code suivant au modèle `mon_foyer.foyer_room` : Comme ci dessous (apres ligne rouge) :

-----help="Select Foyer room equipements")

```
_sql_constraints = [  
    ("room_no_unique", "unique(room_no)", "Le numéro de chambre doit être unique !")  
]
```

2. Contrainte Python

- Une contrainte Python est une méthode qui vérifie une condition sur un ensemble d'enregistrements.
- On utilise le décorateur `@api.constrains()` pour :
 - Marquer la méthode comme une contrainte,
 - Spécifier les champs déclenchant la vérification.
- La contrainte s'exécute automatiquement dès que l'un de ces champs est modifié. Si la condition n'est pas respectée, la méthode doit lever une exception `ValidationError` :

```
from odoo.exceptions import ValidationError  
from odoo import api, _
```

```
@api.constrains("rent_amount")  
def _check_rent_amount(self):  
    """Vérifie que le montant du loyer n'est pas négatif"""  
    if self.rent_amount < 0:  
        raise ValidationError(_("Le montant du loyer mensuel ne peut pas être négatif !"))
```

Mise à jour nécessaire

Après ces modifications dans le fichier de code, il faut :

1. Mettre à jour le module (`Upgrade` dans l'interface Odoo),
2. Redémarrer le serveur (si nécessaire pour certaines configurations).

Resumé

- Nous pouvons utiliser du code Python pour valider nos modèles et empêcher les données invalides. Pour cela, nous utilisons deux éléments :
-
- 1. Une méthode de vérification :
 - - Elle contrôle une condition sur un ensemble d'enregistrements.
 - - Le décorateur `@api.constrains()` indique qu'il s'agit d'une contrainte et précise les champs déclencheurs.
 - - La vérification s'effectue automatiquement dès que l'un de ces champs est modifié.
- 2. Une exception `ValidationError` :
 - - Elle est levée si la condition n'est pas respectée.
 - - Elle affiche un message d'erreur à l'utilisateur et interrompt l'opération en cours.
- - `@api.constrains` garantit que la méthode s'exécute à chaque modification des champs surveillés.
- - `ValidationError` bloque la sauvegarde et informe l'utilisateur de l'erreur.
- - `any()` est utilisé pour vérifier tous les enregistrements concernés (utile en cas d'opérations groupées).

Ajout des champs calculés à un modèle

Nous pouvons avoir besoin de créer un champ qui dépend des valeurs d'autres champs du même enregistrement ou d'enregistrements liés.

- Par exemple, nous pouvons calculer un montant total en multipliant un prix unitaire par une quantité. Dans les modèles Odoo, nous utilisons des champs calculés pour cela.
- Pour montrer comment fonctionnent les champs calculés, nous allons en ajouter un champ au modèle foyer_room
- Et qui calcule la disponibilité des chambres en fonction de l'occupation par les étudiants.
- Nous pouvons aussi rendre ces champs calculés modifiables et recherchables.
- Dans un exemple. Nous allons effectuer des modifications du fichier models/foyer_room.py

1- Les champs calculés

Nous allons modifier le fichier `models/foyer_room.py` pour ajouter un nouveau champ calculé et les méthodes implémentant sa logique :

- Principe des champs calculés :
 - La valeur d'un champ calculé dépend généralement d'autres champs du même enregistrement.
 - L'ORM requiert que le développeur déclare ces dépendances en utilisant le décorateur `@api.depends()` sur la méthode de calcul.
 - L'ORM utilise ces dépendances pour recalculer le champ chaque fois qu'une de ses dépendances est modifiée.
- Nous ajoutons les nouveaux champs au modèle `mon_foyer room` :
- Premier Bloc

help="Select Foyer room equipments")

- #Ajout des champs calculés à un modèle
- `student_per_room = fields.Integer("Student Per Room", required=True,`
- `help="Nombre d'étudiants alloués par chambre")`
- `availability = fields.Float(compute="_compute_check_availability",`
- `store=True, string="Availability", help="Disponibilité de la chambre dans le foyer")`
- ----Deuxieme bloc-----
- **_name = "foyer.room"**
- **_description = "Foyer Room Information"**
- **_rec_name = "room_no"**
- ##Ajout des champs calculés à un modèle
- `@api.depends("student_per_room", "student_ids")`
- `def _compute_check_availability(self):`
- `"""Méthode pour vérifier la disponibilité des chambres"""`
- `for rec in self:`
- `rec.availability = rec.student_per_room - len(rec.student_ids.ids)`
- # fin #Ajout des champs calculés à un modèle

2- Champs calculés modifiables

- Par défaut, les champs calculés sont en lecture seule, car leur valeur est déterminée automatiquement et ne devrait pas être saisie manuellement.
- Cependant, dans certains cas, il peut être utile d'autoriser l'utilisateur à modifier directement la valeur.
- Par exemple, dans notre scénario de gestion de foyer étudiant, nous pourrions ajouter : dans **le modele foyer_student**
 - Date d'admission
 - Date de sortie
 - Durée du séjour
- L'objectif est que l'utilisateur puisse :
 - Soit saisir la durée → la date de sortie se calcule automatiquement.
 - Soit saisir la date de sortie → la durée se met à jour en conséquence.
- Ajouter le code dans le modele mon_foyer Student

2- Champs calculés modifiables

Utilisateur crée un étudiant

↓

Saisit admission_date = 2024-01-01

↓

Saisit discharge_date = 2024-01-31

↓

→ compute calcule duration = 30 jours

↓

Utilisateur modifie duration = 45 jours

↓

→ inverse recalcule discharge_date = 2024-02-15

↓

duration est maintenant incohérent avec la nouvelle discharge_date ?

↓

→ compute se redéclenche et recalcule duration = 45 jours

2- Champs calculés modifiables

1. Utilisateur modifie duration de 30 → 45 jours

↓

2. `_inverse_duration` s'exécute

↓

3. Calcule `duration_actuelle` = 30 (basée sur `discharge_date` actuelle)

↓

4. Compare : 30 != 45 → VRAI

↓

5. Modifie `discharge_date` (+15 jours)

↓

6. `_compute_check_duration` se déclenche

↓

7. Recalcule `duration` = 45 (basée sur nouvelle `discharge_date`)

↓

8. `_inverse_duration` se déclenche à nouveau ???

↓

9. Calcule `duration_actuelle` = 45 (basée sur nouvelle `discharge_date`)

↓

10. Compare : 45 != 45 → FAUX

↓

11. Ne fait rien → BOUCLE ARRÊTÉE !!!

2- Champs calculés modifiables

- Ajouter le code dans le modele mon_foyer Student apres la ligne en rouge existante :

- Premier Bloc :

string="Status", copy=False, default="draft", help="État de l'étudiant dans le foyer")

```
string="Status", copy=False, default="draft", help="État de l'étudiant dans le foyer")
```

```
admission_date = fields.Date("Admission Date",  
                             help="Date d'admission à l'internat",  
                             default=fields.Datetime.today)
```

```
discharge_date = fields.Date("Discharge Date",  
                             help="Date à laquelle l'élève est sorti")
```

```
duration = fields.Integer("Duration", compute="_compute_check_duration", inverse="_inverse_duration",  
                          help="Saisir la durée du séjour")
```

- Deuxieme Bloc :

description = "Foyer Student Information"

```
####Champs calculés modifiables#####
```

```
@api.depends("admission_date", "discharge_date")
```

```
def _compute_check_duration(self):
```

```
    """Méthode pour vérifier la durée"""
```

```
    for rec in self:
```

```
        if rec.discharge_date and rec.admission_date:
```

```
            rec.duration = (rec.discharge_date - rec.admission_date).days
```

```
def _inverse_duration(self):
```

```
    for stu in self:
```

```
        if stu.discharge_date and stu.admission_date:
```

```
            duration = (stu.discharge_date - stu.admission_date).days
```

```
            if duration != stu.duration:
```

```
                stu.discharge_date = (stu.admission_date + timedelta(days=stu.duration)).strftime('%Y-%m-%d')
```

2- Champs calculés modifiables

Méthodes Compute et Inverse

- **Méthode Compute (@api.depends)**

- Rôle : Calcule la valeur du champ basée sur ses dépendances.
- Déclenchement : S'exécute automatiquement dès qu'un des champs listés dans @api.depends() est modifié.

- **Méthode Inverse (inverse=)**

- Rôle : Permet une saisie manuelle du champ calculé en propageant la valeur vers ses dépendances.
- Déclenchement : S'exécute uniquement lors de l'enregistrement du record si l'utilisateur modifie la valeur du champ.

3- Stoker Champs calculés modifiables avec l'attribut store=True

Les champs calculés ne sont pas stockés dans la base de données par défaut. Une solution consiste à stocker le champ en utilisant l'attribut store=True : **dans le modele Foyer Room :**

```
student_per_room = fields.Integer("Student Per Room", required=True,
```

```
    help="Nombre d'étudiants alloués par chambre")
```

```
availability = fields.Float(compute="_compute_check_availability",
```

```
    store=True, string="Availability", help="Disponibilité de la chambre dans le foyer")
```

- Comme les champs calculés ne sont pas stockés dans la base de données par défaut,
- il n'est pas possible d'effectuer une recherche sur un champ calculé, sauf si l'on utilise l'attribut store=True ou que l'on ajoute une méthode de recherche.

Le fonctionnement

- Un champ calculé ressemble à un champ classique, sauf qu'il possède un attribut `compute`` qui spécifie le nom de la méthode utilisée pour calculer sa valeur.
- Cependant, les champs calculés ne sont pas identiques aux champs classiques en interne. Ils sont calculés dynamiquement à l'exécution, et pour cette raison, `ils ne sont pas stockés dans la base de données`. Ainsi, `il n'est pas possible de faire des recherches ou des écritures dessus par défaut`. Il faut effectuer des manipulations supplémentaires pour permettre la modification et la recherche sur ces champs. Voyons comment faire cela.
- La méthode de calcul est exécutée dynamiquement à l'exécution, mais l'ORM utilise un `système de cache` pour éviter de recalculer la valeur à chaque fois qu'on y accède. Pour cela, il a besoin de savoir sur quels autres champs il dépend. On utilise donc le `décorateur `@depends`` pour indiquer quand les valeurs mises en cache doivent être invalidées et recalculées.
- Assurez-vous que la méthode de calcul attribue `toujours` une valeur au champ calculé. Sinon, une erreur se produira. Cela peut arriver si certaines conditions dans votre code empêchent d'attribuer une valeur au champ. Ce type d'erreur peut être difficile à déboguer.
- La prise en charge de l'écriture peut être ajoutée en implémentant la `méthode `inverse``. Celle-ci permet d'utiliser la valeur attribuée au champ calculé pour mettre à jour les champs sources. Bien sûr, cela ne fonctionne que pour des calculs simples, mais dans certains cas, cela peut s'avérer utile.

Le fonctionnement

- Dans notre exemple, cela permet de **définir la date de sortie** en modifiant le nombre de jours de durée, car **duration** est un champ calculé.
- L'attribut **inverse** est facultatif : si vous ne souhaitez pas rendre le champ calculé modifiable, vous pouvez ne pas le définir.
- Il est aussi possible de rendre un champ calculé **non stocké** consultable en définissant l'attribut **search** avec le nom d'une méthode (similaire à `compute`` et `inverse``). Comme `inverse``, **search** est aussi facultatif : si vous ne souhaitez pas rendre le champ consultable, vous pouvez l'ignorer.
- Cependant, cette méthode ne réalise **pas la recherche elle-même**. Elle reçoit l'opérateur et la valeur utilisés pour la recherche comme paramètres, et **doit retourner un domaine** (`domain``) contenant les conditions de recherche alternatives à utiliser.
- L'option **store=True** permet de **stocker le champ dans la base de données**. Dans ce cas, une fois le champ calculé, sa valeur est conservée en base, et elle sera ensuite récupérée comme un champ classique, **au lieu d'être recalculée à chaque fois**.
- Grâce au décorateur **@api.depends**, l'ORM saura quand ces valeurs stockées doivent être recalculées et mises à jour. Vous pouvez considérer cela comme un **cache persistant**.
- Cela permet aussi d'utiliser ce champ dans les **recherches, tris et regroupements**. Si vous utilisez `store=True`` sur votre champ calculé, **vous n'avez plus besoin d'implémenter la méthode `search``**, car le champ est désormais en base de données et peut être utilisé normalement pour les recherches/tri.
- L'option **compute_sudo=True** est utile lorsque le calcul du champ nécessite des **droits d'accès élevés**. Cela peut être nécessaire lorsque les données utilisées dans le calcul ne sont **pas accessibles par l'utilisateur final**.

- **Points importants :**
- Modification manuelle : Vous pouvez maintenant modifier duration dans l'interface Odoo, et discharge_date sera automatiquement recalculé.
- Store=True : J'ai ajouté store=True pour que la valeur calculée soit stockée en base et puisse être utilisée dans les recherches et filtres.
- Validation des dates : J'ai ajouté une contrainte pour empêcher les dates incohérentes.
- Gestion des erreurs : Meilleure gestion des cas où les dates ne sont pas définies.
- Utilisation :
- Calcul automatique : Si vous modifiez admission_date ou discharge_date, duration se recalcule
- Modification manuelle : Si vous modifiez duration, discharge_date se recalcule automatiquement
- Votre approche avec compute + inverse est correcte pour permettre la modification manuelle d'un champ calculé !

Le fonctionnement

```
<group col="4">
```

```
  <field name="student_per_room"/>
```

```
  <!-- Ceci est un commentaire -->
```

```
  <field name="availability"/>
```

```
  <field name="rent_amount"/>
```

```
  <field name="currency_id"/>
```

```
</group>
```

Tester le fonctionnement

- Comment tester le fonctionnement :
- 1-Créer une chambre:
- 2-Définissez "Student Per Room" à 4 (par exemple)
- 3-L'"Availability" devrait montrer 4 (car aucun étudiant n'est encore assigné)
- 4-Ajouter des étudiants: Allez dans l'onglet "Students" du formulaire de la chambre ,Ajoutez 2 étudiants
- L'"Availability" devrait automatiquement passer à 2 ($4 - 2 = 2$)
- Modifier la capacité:
- Si vous changez "Student Per Room" à 3 avec 2 étudiants déjà présents
- L'"Availability" devrait passer à 1 ($3 - 2 = 1$)

fin